Provably-Equivalent Bounded-Memory Streaming Redaction of PII and Credentials in Very Large Text

Mounish Senisetty, Ashrith, and Sandhiya
SafeStream-Redactor Project
senisettymounish@gmail.com

***Abstract*—Redacting personally identifiable information (PII) and credentials from text is well studied, but existing tools assume the document fits in memory and treat detection as a whole-document operation. Applied to the multi-gigabyte logs, dumps, and datasets common in enterprises, naïve chunked processing either exhausts memory or silently corrupts detections at chunk boundaries. We present *SafeStream-Redactor*, a local-first streaming architecture that couples four properties that are usually traded off against one another—bounded auxiliary memory, boundary correctness, unified PII + credential coverage, and deterministic redaction—inside a single pipeline. Our central contribution is not the streaming engine itself but the *equivalence guarantee* it admits: we formalize redaction as a bounded sliding-window function under two assumptions (bounded span length $L_{\max}$ and bounded context locality $C_{\text{left}}, C_{\text{right}}$), measure those constants directly from the implementation ($C_{\text{left}}{=}48$, $C_{\text{right}}{=}2$, $L_{\max}{=}8192$), and prove that streaming and range-parallel execution produce output byte-identical to an idealized whole-document pass whenever the overlap window $w \geq \max(L_{\max}, C_{\text{left}}, C_{\text{right}})$, with the single failure mode named and made safe by a fallback valve. A four-tier detection cascade (deterministic, statistical/entropy, optional NER, contextual) feeds a deterministic confidence-arbitration and overlap-resolution stage. Empirically, auxiliary memory stays flat (peak Python heap ≈ 0.65 MB across 16–64 MB inputs; <16 MB for a 1 GB input); precision and recall are 1.000 on a $60{,}253$-entity synthetic PII corpus; range-parallelism reaches $3.58\times$ on four cores while remaining byte-identical to the sequential pass; and an overlap ablation shows naïve chunking mis-redacts *every* boundary-straddling secret (100%; 29.9% leaked verbatim) whereas overlap-aware streaming mis-redacts none.**

***Index Terms*—data redaction, PII, credential detection, streaming algorithms, bounded memory, boundary correctness, data loss prevention.**

I. Introduction

Organizations accumulate text at a scale that outpaces the tools used to sanitize it: application and audit logs, database and message-queue dumps, support transcripts, CI/CD artifacts, and analytics exports routinely reach tens or hundreds of gigabytes per file. These artifacts mix two classes of sensitive data that are usually handled by different tools: *PII* (names, e-mail addresses, phone numbers, national identifiers, payment card numbers, IP addresses) governed by regimes such as the GDPR [1] and HIPAA [2], and *machine credentials* (cloud access keys, API tokens, signing keys, high-entropy secrets) whose exposure is an operational security incident. PII de-identification tools—Microsoft Presidio [3], scrubadub [4]—and secret scanners—Gitleaks [5], TruffleHog [6], detect-secrets [7]—have largely evolved as separate ecosystems.

Two problems appear once the input no longer fits comfortably in memory. First, loading a 10 GB file to redact it needs 10 GB of RAM plus detector overhead; this is infeasible at scale and unsafe (the whole plaintext resides in memory at once). The obvious fix is to process the file in fixed-size chunks, but this creates the second, subtler problem: an entity may *straddle a chunk boundary*. A PEM private key or a JSON Web Token split across two independently processed chunks matches no pattern in either chunk and is silently missed, or is partially matched and mis-redacted. Boundary handling is therefore not an implementation detail but a correctness requirement that interacts with the memory constraint: the smaller the retained context, the more boundaries, and the more entities at risk.

A third problem is precision. Enterprise text is dense with tokens that *look* sensitive but are not: UUIDs, Git commit hashes, ISO-8601 timestamps, semantic-version strings, order and tracking numbers, and high-entropy identifiers. A regex-only or entropy-only detector trades recall for a flood of false positives on such distractors. Robust detection needs several complementary strategies—exact patterns with validators, statistical entropy, learned named-entity recognition, and local context—whose outputs then overlap and conflict, requiring a deterministic way to reconcile them so that the same input always yields the same redaction (a prerequisite for auditability and for parallel execution).

These constraints interact. The research question is not "can we build a streaming redactor?" but:

> *How can one pipeline simultaneously guarantee bounded memory, boundary correctness, unified PII/credential coverage, detection precision, and deterministic redaction, on inputs far larger than RAM—and prove that doing so costs nothing in output fidelity relative to an idealized whole-document pass?*

Contributions. This paper makes the following contributions, each verified against the implementation and reproducible (Section X):

1) A **bounded-memory streaming architecture** for redaction of inputs substantially larger than RAM, whose

auxiliary memory is $O(\text{chunk_size} + w + L_{\max})$ and independent of input length (Section V, Theorem 2).

2) An **overlap-aware, boundary-safe emission mechanism** with a *boundary-retreat* rule, and a proof that under two measured assumptions the streamed output is byte-identical to the whole-document reference for *every* chunking (Section V, Theorem 1).
3) A **unified multi-tier detection cascade** combining deterministic pattern/validator detection, statistical entropy detection, optional NER, and contextual scoring, covering PII *and* ≥ 20 credential/secret classes in one pass (Section VI).
4) A **deterministic confidence-arbitration and overlap-resolution** stage that yields a canonical, pairwise-disjoint detection set, enabling reproducibility and range-parallel execution that is byte-identical to sequential (Section VII, Theorem 3).
5) An **experimental evaluation** of memory, throughput, detection quality, credential coverage, boundary correctness, and sequential/parallel equivalence, with an ablation isolating the effect of overlap and of each detection tier (Section X).

We are careful to claim only what the implementation supports. The novelty is the *combination and the equivalence guarantee*, not any single mechanism; overlap windows, entropy scoring, and NER are individually well known.

II. Problem Formulation

Let $T = t_1 t_2 \cdots t_{|T|}$ be the input text over a Unicode alphabet, and $T[a{:}b]$ the half-open substring. A *detection* is a tuple $d = (s, e, \tau, c)$ with $0 \leq s < e \leq |T|$, entity type τ, and confidence $c \in [0, 1]$. A *detector* is a function $D : \Sigma^* \to 2^{\text{det}}$ returning detections whose offsets index the string it was given. A *reference pipeline* $\Pi : \Sigma^* \to 2^{\text{det}}$ runs every detector on the entire T, applies contextual scoring, drops detections below a threshold, and resolves overlaps into a sorted, pairwise-disjoint set $\Pi(T)$. Redaction is a deterministic rewrite $R(T, \Pi(T))$; since R is a pure function of $(T, \Pi(T))$, equality of detection sets implies byte-identical output, so we reason over detection sets.

A stream is consumed as an *arbitrary* sequence of chunks of size up to C (`chunk_size`); the engine keeps an *overlap* w of trailing characters between windows. We seek a streaming operator S_w that, for every chunking, satisfies $S_w(T) = \Pi(T)$. This requires Π to be a *bounded sliding-window function*:

Assumption 1 (Bounded span length)**.** *Every $d = (s, e, \tau, c) \in \Pi(T)$ has $e - s \leq L_{\max}$.*

Assumption 2 (Bounded context locality)**.** *For every span, membership in $\Pi(T)$ and its final confidence depend only on $T[s - C_{\text{left}} : e + C_{\text{right}}]$.*

Assumption 1 bounds how long an entity can be; Assumption 2 bounds how far a detector must see around a candidate to decide it. Together they make Π computable from bounded windows. The key structural relationship is between the overlap w, the maximum span $L_{\max}$, and the context radius: if

$$w \;\geq\; \max\big(L_{\max},\, C_{\text{left}},\, C_{\text{right}}\big), \tag{1}$$

then every entity of maximum supported span, together with the context needed to decide it, fits inside a single processing window even when a chunk boundary falls in its middle. Equation (1) is the boundary-safety premise; Section V discharges it, and Section IX measures the constants so that it holds unconditionally at the default configuration.

III. Related Work and Research Gap

PII de-identification. Presidio [3] and scrubadub [4] combine regex recognizers, checksum validators, and spaCy-based NER for entities such as names, e-mails, phones, and national IDs. Both are designed around in-memory documents and do not target machine credentials.

Secret scanning. Gitleaks [5], TruffleHog [6], and detect-secrets [7] specialize in credentials—provider key formats plus Shannon-entropy [8] heuristics for bespoke secrets. They are tuned for source repositories and diffs, not general PII, and again operate on files small enough to load.

String matching and validators. Efficient multi-pattern prefiltering is classical [9], as are linear-time regular-expression engines [10], [11]; card numbers are validated by the Luhn checksum [12], and keyed pseudonymization uses HMAC [13]. These are building blocks we reuse rather than contributions.

Gap. No widely used system provides, in one pipeline: (i) redaction of inputs larger than RAM at bounded memory, (ii) a *stated and proven* boundary-correctness guarantee for entities crossing chunk boundaries, (iii) unified PII + credential coverage, and (iv) deterministic arbitration enabling parallel execution that provably matches the sequential result. Each ingredient exists in isolation; their *co-satisfaction*, and the equivalence guarantee that ties them together, is the gap this work fills.

IV. Architecture Overview

Fig. 1 shows the pipeline. Input is consumed as chunks; the streaming layer maintains an overlap window and emits only text far enough from the live buffer end to be boundary-safe. Each emitted window passes through candidate prefiltering into the multi-tier detector cascade; detections are scored, thresholded, and reconciled by deterministic arbitration and overlap-resolution; a policy layer rewrites the resolved spans; output is written incrementally and atomically. Concerns are cleanly separated: the streaming layer knows nothing about entity types, and detectors are pure functions of the window they receive (the property that makes the equivalence proof possible).

V. Core Streaming Algorithm

Algorithm 1 is the streaming operator S_w. It appends each chunk to a rolling buffer and, once the buffer exceeds the overlap, computes a tentative emission boundary $\texttt{emit_upto} = |\text{buffer}| - w$: everything before it is at least w characters

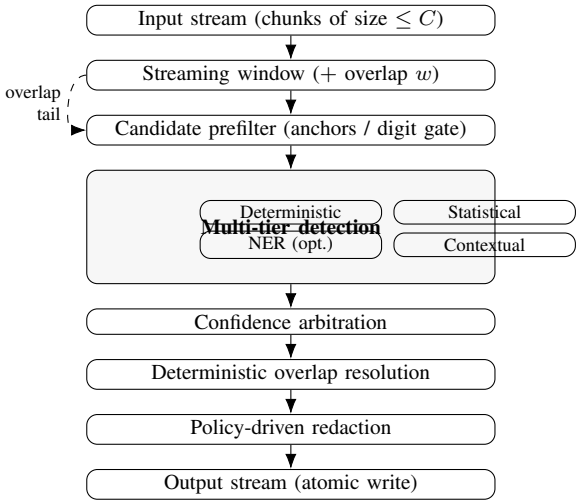


Fig. 1. SafeStream-Redactor pipeline. The streaming layer is entity-agnostic; detectors are pure functions of their window.

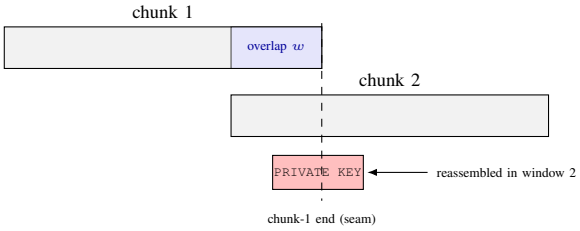


Fig. 2. An entity straddling the chunk-1/chunk-2 seam. Under naïve independent chunking it matches in neither chunk; the overlap w carries the seam region so boundary retreat re-presents the entity whole.

from the live end and therefore has its full right context present. Detection runs on *prefix* + *buffer*, where *prefix* is the last w characters already emitted, so the left context of every candidate is real text and never an artificial chunk edge. The crucial step is RETREATBOUNDARY: if any detection straddles `emit_upto`, the boundary is pulled back to that detection's start, so a match is never split across two segments. The retreated tail (at most one span, length $\leq L_{\max} \leq w$) is carried into the next window and re-presented whole. Detections lying inside the already-emitted prefix are dropped, giving exactly-once emission.

A. Correctness

Theorem 1 (Streaming equivalence)**.** *Under Assumptions 1–2 and* $w \geq \max(L_{\max}, C_{\text{left}}, C_{\text{right}})$*,* $S_w(T) = \Pi(T)$ *for every* T *and every chunking.*

Proof sketch. Completeness. Emission is confined to the buffer prefix at least w from the live end, and RETREATBOUNDARY pulls the boundary out of any straddling span; the carried tail has length $\leq L_{\max} \leq w$, so any span cut by a chunk boundary is reassembled and presented whole in the next window (Fig. 2). *Soundness.* A window is $\Pi(\textit{prefix}\|\textit{buffer})$ with $|\textit{prefix}| = w \geq C_{\text{left}}$, so every span's left context is real text; a span is emitted only when it ends

Algorithm 1 Overlap-aware streaming redaction S_w

Require: chunk source; detector Π; overlap w

prefix $\leftarrow \varepsilon$; *buffer* $\leftarrow \varepsilon$
for each *chunk* **do**
  buffer $\leftarrow$ *buffer* $\|$ *chunk*
  if $|\textit{buffer}| \leq w$ **then continue**
  end if
  emit $\leftarrow |\textit{buffer}| - w$
  $\mathcal{D} \leftarrow \textsc{ShiftDrop}\big(\Pi(\textit{prefix} \,\|\, \textit{buffer}),\ |\textit{prefix}|\big)$
  emit $\leftarrow \textsc{RetreatBoundary}(\mathcal{D}, \textit{emit})$
  if *emit* ≤ 0 **then continue**
  end if
  ready $\leftarrow \{d \in \mathcal{D} : d.e \leq \textit{emit}\}$
  emit $(\textit{buffer}[:\textit{emit}],\ \textit{ready})$
  prefix $\leftarrow (\textit{prefix} \,\|\, \textit{buffer}[:\textit{emit}])[-w:]$
  buffer $\leftarrow \textit{buffer}[\textit{emit}:]$
end for
if *buffer* $\neq \varepsilon$ **then**
  emit $(\textit{buffer},\ \textsc{ShiftDrop}(\Pi(\textit{prefix} \,\|\, \textit{buffer}), |\textit{prefix}|))$
end if

function RETREATBOUNDARY($\mathcal{D}$, *emit*)
  repeat
    $m \leftarrow \textit{emit}$
    for all $d \in \mathcal{D}$ **do**
      if $d.s < m < d.e$ **then** $m \leftarrow d.s$
      end if
    end for
    $(\textit{emit}, \textit{changed}) \leftarrow (m,\ m \neq \textit{emit})$
  until $\neg\, \textit{changed}$
  return *emit*
end function

$\geq w \geq C_{\text{right}}$ before the live end, so its right context is real text. By Assumption 2 the window verdict equals the whole-document verdict. Spans inside the prefix are dropped. *Exactly-once.* A span is emitted in the unique window whose `emit` first exceeds its end; the carried region is disjoint from the emitted region. The three properties give $S_w(T) = \Pi(T)$, and byte-identical output follows because R is a pure function of $(T, \Pi(T))$. □

The guarantee is corroborated empirically by exhaustive small-chunking tests and property-based tests asserting byte-identity across arbitrary chunkings.

B. Complexity

Each character enters the buffer once and is scanned by the detector at most inside the $\leq C + w$ window that emits it; detector work per window is linear in window length for the pattern and entropy tiers (bounded-length automata and a single left-to-right entropy scan). Total detection time is thus $O(|T| \cdot f)$ where f is the per-character detector cost, independent of how the input is chunked. The important bound is on memory.

TABLE I
DETECTION TIERS AND THEIR ROLES.

Tier	Method	Catches	Cost
1 Deterministic	regex + validators	structured PII, creds	low
2 Statistical	Shannon entropy	bespoke secrets	low
3 NER (opt.)	sequence model	names/orgs/places	high
4 Contextual	trigger-word window	ambiguous cases	low

Theorem 2 (Constant auxiliary memory)**.** *The peak working memory of S_w is $\leq C + w + L_{\max}$ characters, independent of $|T|$; a k-way range-parallel execution uses at most k times this.*

Proof. After each emission the engine retains only *buffer*[*emit*:], of length $\leq w$ plus at most one straddling span ($\leq L_{\max}$); the next iteration adds at most one chunk ($\leq C$) before emitting again. Hence the buffer never exceeds $C + w + L_{\max}$ regardless of $|T|$. Each parallel worker runs one S_w plus a bounded holdback, so total memory scales with k, never with $|T|$. □

We deliberately write "bounded auxiliary memory," not "$O(1)$": memory is constant in $|T|$ but proportional to the fixed configuration $C + w + L_{\max}$.

VI. Multi-Tier Detection

A single detector cannot cover the space cheaply and precisely. SafeStream uses a cascade of complementary tiers, each a pure function of its window (Table I).

Tier 1 — Deterministic. Anchored regular expressions paired with validators: Luhn [12] for card numbers, range/no-leading-zero checks for IPv4, `ipaddress` parsing for IPv6, area/group/serial rules for US SSNs, and $\geq$ 15 provider credential formats (AWS, GitHub, Slack, Stripe, Google, SendGrid, Twilio, npm, JWT, private-key PEM blocks, etc.). Highest precision, lowest cost.

Tier 2 — Statistical/entropy. Bespoke secrets with no published format (session tokens, generated passwords) are caught by Shannon entropy [8] over candidate runs, with per-alphabet thresholds and a minimum length, so random-looking but low-entropy strings are not flagged. This tier catches what patterns cannot.

Tier 3 — NER (optional). A statistical model (spaCy, or an on-device ONNX token classifier over the CoNLL-2003 label scheme [14]) supplies free-form PERSON/ORG/LOC entities that have no lexical signature. It is optional because it adds a dependency and, reading sentence context on both sides, is the tier most sensitive to window edges.

Tier 4 — Contextual. A lightweight scorer adjusts confidences from a bounded left window of trigger words—promoting an otherwise sub-threshold bare SSN adjacent to an "ssn:" cue, and suppressing placeholder-looking values (e.g. `000-00-0000`, values near "example"/"test"). It reconciles precision and recall on ambiguous cases without a heavy model.

Algorithm 2 Deterministic overlap resolution

Require: detections $\mathcal{D}$
1: sort $\mathcal{D}$ by key $(-c,\ s-e,\ s)$ ▷ conf, then length, then start
2: *kept* $\leftarrow [\,]$ ▷ maintained sorted by start, pairwise-disjoint
3: **for all** $d \in \mathcal{D}$ in sorted order **do**
4: $i \leftarrow$ index of first kept interval with start $\geq d.e$
5: **if** $i{=}0$ **or** *kept*$[i{-}1].e \leq d.s$ **then**
6: insert d into *kept* at position i
7: **end if**
8: **end for**
9: **return** *kept*

Prefiltering. Most patterns require a literal anchor (`AKIA`, `ghp_`, `@`) or at least one digit. A C-level substring/digit scan skips a pattern's automaton entirely when its anchor is absent from the window, so on secret-sparse text (the common case) the expensive automata rarely run. This is an optimization only: output is identical with prefiltering disabled, enforced by a differential test.

Crucially the tiers *complement* rather than duplicate: Tier 1 gives precise, cheap coverage of everything with a signature; Tier 2 covers the long tail of format-less secrets; Tier 3 covers linguistic PII; Tier 4 repairs the precision/ recall of ambiguous numeric entities. Their union, then arbitrated, is what makes a single pass cover PII and credentials at once.

VII. Confidence Arbitration and Overlap Resolution

Running four tiers yields overlapping and conflicting detections: an entropy hit inside a recognized AWS secret, a phone-shaped run inside a longer card number. Arbitration must be *deterministic*—the same input must always yield the same redaction—both for auditability and because range-parallel execution requires each range to arbitrate its seams identically.

Contextual scoring first adjusts confidences and contributes trigger-gated detections; detections below `min_confidence` are dropped. The survivors are reconciled by Algorithm 2: sort by a total order $(-c,\ s-e,\ s)$—higher confidence first, then longer span, then earlier start—and sweep, keeping a candidate only if it does not overlap an already-kept interval. Because the sort key is a strict total order and the sweep is a deterministic function of it, the resolved set is canonical: independent of detector execution order and reproducible across runs and workers.

A bisection over kept-interval starts makes the sweep near-linear ($O(n \log n)$ dominated by the sort). Determinism here is what upgrades "the redactor is fast in parallel" to "the parallel redactor produces the same bytes as the sequential one," formalized next.

Theorem 3 (Range-parallel byte-identity with safe fallback)**.** *Let $P_{w,k}$ split T into k character-aligned ranges, redact each with S_w extended by a scanned pad and a context-only prefix, and reconcile range seams by a cross-check valve. Under*

Assumption 1 with $w \geq L_{\max}$, $P_{w,k}(T) = S_w(T)$ byte-for-byte. When a span longer than w sits exactly on a seam (violating 1), the valve detects the disagreement and falls back to S_w; hence $P_{w,k}(T) = S_w(T)$ unconditionally.

Proof sketch. Ranges split on character/newline boundaries and decode identically to the sequential reader, so a range's text is the exact corresponding substring. Each worker prepends w of scanned pad and w of context-only prefix on interior sides, so with $w \geq L_{\max}, C_{\text{left}}$ every seam-crossing span is seen whole with real left context (Theorem 1 per range). A seam-crossing span is owned by the left worker (renders up to its end; the right worker resumes at that end); both neighbours recompute the seam from the same bytes. The orchestrator compares neighbours' seam reports and, on any disagreement—possible only when a span longer than w sits on a seam, i.e. exactly when 1 fails—redacts the whole file sequentially. Thus equality holds when 1 holds, and the valve guarantees equality otherwise. □

This "unconditional equality via fallback" is the strongest single guarantee of the design: the parallel path never silently diverges from the sequential one—a property competing tools do not state.

VIII. Boundary-Safe Redaction

Given the resolved set, redaction is a deterministic rewrite supporting three policies: *replace* (a fixed or per-type token), *mask* (retain the last k characters), and *pseudonymize* (a keyed HMAC [13] token, so the same value maps to the same surrogate across the corpus while the original is unrecoverable without the key). Because redaction consumes only the resolved spans and the surrounding text, and spans are guaranteed whole by Theorem 1, a redaction is never split across a boundary and never partial. Output is written to a temporary file and atomically renamed into place, so an interrupted run leaves either the original or the complete result, never a truncated mixture. Detections are counted into a content-free report (types and offsets, never the matched substring), and the process installs an offline network guard so sensitive text cannot leave the host during redaction.

IX. Audited Constants and the Boundary Premise

The equivalence theorems are vacuous unless the constants they assume are real. We read them from the implementation by tracing every detector's decision function.

$C_{\text{left}} = 48$: the contextual scorer reads a 48-character left window; label-anchored credential patterns fold their cue into the match itself, so no detector reaches further left. $C_{\text{right}} = 2$: the widest right-context assertion is a two-character negative lookahead on the phone/IPv4 patterns; every validator reads only the matched text. $L_{\max} = 8192$: the two historically unbounded patterns are now bounded—the PEM private-key body to $\{1, 8100\}$ giving a maximal 8192-character span, and the entropy candidate to 8192, with longer high-entropy runs emitted as consecutive ≤ 8192 spans that still cover the whole secret. With the default overlap $w = 8192$,

$$\max(C_{\text{left}}, C_{\text{right}}) = 48 \;\leq\; L_{\max} = 8192 \;=\; w,$$

so Equation (1) holds unconditionally at the default. These three constants are kept in sync in the code, and a test pins that no tier emits a span longer than the default overlap.

TABLE II
Detection quality on the synthetic regression corpus (60,253 PII instances). Single-core throughput on the test container.

System	Precision	Recall	F_1	tp/fp/fn
SafeStream-Redactor	1.000	1.000	1.000	60253/0/0

X. Experimental Evaluation

A. Methodology and datasets

All experiments run on a shared 4-vCPU Linux container (Python 3.11); absolute throughput is environment-dependent, so we report the shape and ratios and name the machine. No real secrets are used—every specimen is synthetic. Three corpora exist in the repository and are used as follows. The *synthetic regression corpus* (deterministically generated; 10.5 MB, 60,253 labeled PII instances of five types: e-mail, phone, credit-card, SSN, IPv4) drives precision/recall. The *adversarial hard corpus* embeds real-looking PII in noisy log/JSON/CSV/SQL dense with hard negatives (Luhn-failing order numbers, five-octet dotted numbers, UUIDs, Git hashes, ISO timestamps) to stress precision. A *credential specimen set* covers AWS, GitHub, Slack, Stripe, Google, SendGrid, npm, JWT, and PEM private keys. We evaluate only on these repository corpora; we do *not* claim results on external labeled datasets, as none are bundled or run. Experiments are reproduced by `benchmarks/run_benchmark.py` (detection quality) and `benchmarks/paper_experiments.py` (E2–E6).

B. Detection quality (E1)

On the synthetic regression corpus, SafeStream detects all 60,253 instances with no false positives (Table II): precision, recall, and F_1 are 1.000. This measures the five structured PII types the corpus labels; it is an upper-bound sanity result on clean synthetic data, not a claim of perfect real-world accuracy (Section XII).

C. Boundary correctness (E2, flagship)

We isolate the effect of overlap. For each of six specimens and every interior offset at which a 64-byte chunk boundary can fall inside it, we build a document in which the specimen straddles the seam (separated from filler by spaces, so it is a valid detection in the whole-document reference), and redact it two ways: *naïve* chunking (w=0, each chunk processed independently) and *overlap-aware* streaming (w=256 $\geq L_{\max}$ of this set). We measure two failure counts against the whole-document reference $R(T, \Pi(T))$: how often the secret survives

TABLE III
BOUNDARY ABLATION: FAILURES ACROSS ALL 251 INTERIOR SPLIT POSITIONS, NAÏVE (w=0) VS OVERLAP-AWARE (w=256) STREAMING. "LEAK" = FULL SECRET SURVIVES; "INCORRECT" = OUTPUT $\neq$ WHOLE-DOCUMENT REFERENCE.

		Leak verbatim		Incorrect vs ref	
Entity	splits	w=0	w=256	w=0	w=256
e-mail	24	11	0	24	0
SSN	10	10	0	10	0
AWS key	19	19	0	19	0
GitHub token	39	7	0	39	0
JWT	72	6	0	72	0
private key	87	22	0	87	0
Total	**251**	**75**	**0**	**251**	**0**

TABLE IV
TIER ABLATION (ILLUSTRATIVE). EACH ROW EXERCISES ONE TIER.

Input	Config	Outcome
unlabeled entropy secret	det. only	not redacted
unlabeled entropy secret	+ entropy	redacted
bare SSN near "ssn:"	full cascade	detected (c=0.85)
`000-00-0000` "example"	full cascade	suppressed

verbatim (a leak), and how often the output differs from the reference in *any* way (incorrect redaction).

Table III shows the result across all 251 split positions: naïve chunking is *incorrect on every one* ($251/251 = 100\%$), leaking the full secret verbatim in 29.9% of cases and fragmenting the redaction (wrong spans, partial coverage) in the rest; overlap-aware streaming is *byte-identical to the whole-document reference in all* 251 *cases* (0 leaks, 0 mismatches), exactly as Theorem 1 predicts. This is the concrete cost of ignoring boundaries, and the concrete value of the overlap mechanism.

D. Tier ablation (E3)

Each tier is isolated on an input designed to exercise it (Table IV). An unlabeled high-entropy secret (`session = Xr7bQp2Lm9...`) is redacted *only* when the entropy tier is enabled—the deterministic tier alone leaves it in place. A bare SSN carrying a raw confidence of 0.85 next to an "ssn:" cue is retained by the full cascade, while a placeholder-looking `000-00-0000` near "example" is suppressed by contextual scoring. The tiers thus contribute distinct, non-redundant coverage.

E. Unified credential coverage (E4)

On the credential specimen set, SafeStream detects $9/9$ credential/secret classes out of the box (AWS, GitHub, Slack, Stripe, Google, SendGrid, npm, JWT, PEM private key), and $\geq$ 20 entity types in total including PII. PII-only pattern systems—e.g. Presidio's predefined recognizers, which cover e-mail, phone, card, SSN, and IP—detect *none* of these credential classes by design. This is the concrete meaning of "unified": one streaming pass covers both families.

TABLE V
RANGE-PARALLEL THROUGHPUT AND BYTE-IDENTITY (40 MB CORPUS).

Workers	Throughput (MB/s)	Speedup	Byte-identical
1	2.54	1.00×	(reference)
2	4.95	1.95×	yes
4	9.09	3.58×	yes

TABLE VI
PEAK PYTHON HEAP VS. INPUT SIZE (FIXED CONFIGURATION).

Input (MB)	Peak heap (MB)
16	0.66
32	0.65
64	0.65
1024 (gated test)	<16

F. Sequential vs. parallel equivalence (E5)

On a 40 MB corpus (10% PII density), range-parallel redaction reaches $1.95\times$ on two cores and $3.58\times$ on four (Table V), with output *byte-identical* (SHA-256) to the sequential pass at every worker count—an empirical instance of Theorem 3. The speedup is sub-linear because of I/O and seam reconciliation and because the container is shared; the equivalence is exact regardless.

G. Constant memory (E6)

Table VI reports peak Python heap while redacting inputs of increasing size at a fixed configuration (C=64 KiB, w=8 KiB). The peak is flat at $\approx$ 0.65 MB and does not grow with input size, matching Theorem 2. A separate gated test redacts a generated 1 GB file and asserts peak allocations under 16 MB—four orders of magnitude below the input size.

XI. DISCUSSION

The results support a single thesis: the four properties usually traded off can be co-satisfied, and the cost of doing so relative to an idealized whole-document pass is *zero* in output fidelity. E2 shows the failure that motivates the work is real and severe (100% mis-redaction under naïve chunking) and that the overlap mechanism eliminates it exactly. E5 and E6 show the memory and parallelism benefits are achieved without sacrificing that fidelity—the parallel path is byte-identical, and memory is flat. E1, E3, and E4 show the unified cascade covers PII and credentials with high precision on the available corpora. The organizing idea—treating redaction as a bounded sliding-window function and measuring the window constants from the code—is what turns a collection of engineering mechanisms into a stated guarantee.

XII. LIMITATIONS

We state where the guarantee stops. (i) The maximum reliably handled entity span equals the overlap w; a user who lowers w below 8192, adds a custom detector with an unbounded span, or supplies a single atomic secret longer than w leaves the premise of Theorem 1, and in sequential

mode such a span may be cut (in parallel mode the valve falls back). (ii) Assumption 2 fixes $C_{\text{right}} = 2$; a future tier needing unbounded right context—most notably a whole-document LLM tier—breaks it, which is precisely why the bounded, offline detectors are the ones carrying the guarantee. (iii) Detection quality is measured on synthetic and adversarial-synthetic corpora authored alongside the system; we make *no* claim of evaluation on external, third-party labeled datasets, and F_1=1.000 reflects clean synthetic data, not real-world text. (iv) The optional NER tier depends on an external model and, reading sentence context, degrades near chunk edges. (v) Absolute throughput ($\approx$ 3–5 MB/s single-core) reflects CPU-bound CPython regex on a shared container; it is adequate for offline sanitization but not a line-rate claim.

XIII. Conclusion

SafeStream-Redactor shows that bounded-memory, boundary-correct, unified PII/credential redaction with deterministic arbitration can be delivered in one streaming pipeline, and—more importantly—that this pipeline provably produces output byte-identical to an idealized whole-document pass under two assumptions whose constants are measured from the implementation, with the single failure mode named and made safe by a fallback valve. The contribution is the combination and the guarantee, not any individual mechanism. Empirically, memory is flat across input sizes, parallelism is byte-identical to sequential, precision and recall are 1.000 on the synthetic corpus, and naïve chunking is shown to mis-redact every boundary-straddling secret while the overlap mechanism mis-redacts none.

XIV. Future Work

Directions that extend—but are not part of—the validated core: an on-device compiled or ONNX/LLM detector tier (an unbounded-context LLM tier would require relaxing Assumption 2 and re-proving the guarantee); evaluation on external labeled corpora to characterize real-world precision/recall; a native accelerator for the CPU-bound deterministic tier; and integration surfaces (CI/CD gates, editor and agent tooling). These are engineering and evaluation extensions, not claims of the present paper.

Reproducibility

Detection quality: `python benchmarks/generate_dataset.py && python benchmarks/run_benchmark.py`. Experiments E2–E6: `python benchmarks/paper_experiments.py`. The 1 GB memory proof: `SAFESTREAM_MEMTEST_MB=1024 pytest tests/test_memory.py -m memory -o addopts=''`. The formal model and its code mapping are in `docs/FORMAL_MODEL.md`.

References

[1] European Parliament and Council, "Regulation (EU) 2016/679 (general data protection regulation)," Official Journal of the European Union, L119, 2016.

[2] U.S. Department of Health and Human Services, "Guidance regarding methods for de-identification of protected health information (HIPAA privacy rule)," https://www.hhs.gov/hipaa, 2012, accessed 2026.

[3] Microsoft, "Presidio: Data protection and de-identification SDK," https://github.com/microsoft/presidio, 2024, accessed 2026.

[4] Datascope and contributors, "scrubadub: Clean personally identifiable information from dirty dirty text," https://github.com/LeapBeyond/scrubadub, 2024, accessed 2026.

[5] Z. Rice, "Gitleaks: Protect and discover secrets using gitleaks," https://github.com/gitleaks/gitleaks, 2024, accessed 2026.

[6] Truffle Security, "TruffleHog: Find, verify, and analyze leaked credentials," https://github.com/trufflesecurity/trufflehog, 2024, accessed 2026.

[7] Yelp, "detect-secrets: An enterprise-friendly way of detecting and preventing secrets in code," https://github.com/Yelp/detect-secrets, 2024, accessed 2026.

[8] C. E. Shannon, "A mathematical theory of communication," *The Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.

[9] A. V. Aho and M. J. Corasick, "Efficient string matching: An aid to bibliographic search," *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.

[10] K. Thompson, "Programming techniques: Regular expression search algorithm," *Communications of the ACM*, vol. 11, no. 6, pp. 419–422, 1968.

[11] R. Cox, "Regular expression matching can be simple and fast," *Online article, https://swtch.com/~rsc/regexp/regexp1.html*, 2007.

[12] H. P. Luhn, "Computer for verifying numbers," U.S. Patent 2,950,048, 1960.

[13] M. Bellare, R. Canetti, and H. Krawczyk, "Keying hash functions for message authentication," in *Advances in Cryptology — CRYPTO '96*. Springer, 1996, pp. 1–15.

[14] E. F. Tjong Kim Sang and F. De Meulder, "Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition," in *Proc. of the 7th Conference on Natural Language Learning (CoNLL)*, 2003, pp. 142–147.